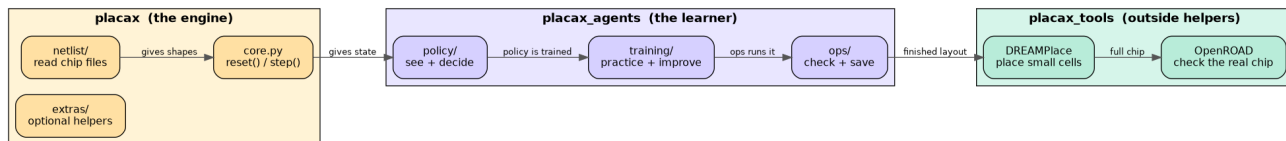


placax

A friendly, whole-codebase tour — every file, every function,
explained like you're five years old

placax is a computer program that plays a game: place the big pieces (macros) of a computer chip onto a grid, as cleverly as possible, so the wires connecting them end up as short as possible. An AI plays this game over and over, getting a little better each time, the same way you'd get better at a video game the more you played it.

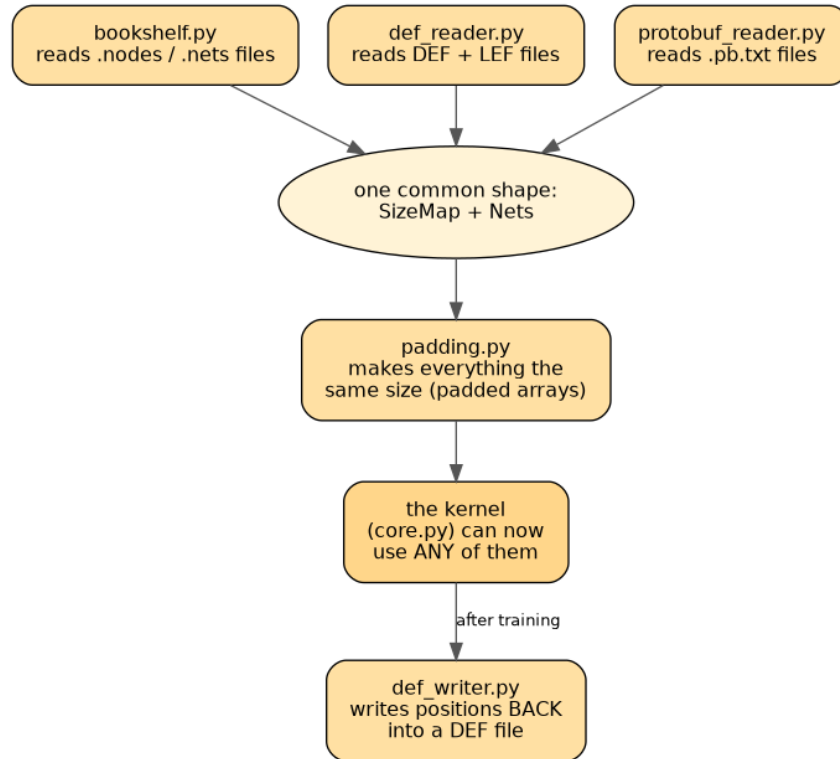


The three packages, and how they hand work to each other.

How the pieces fit together

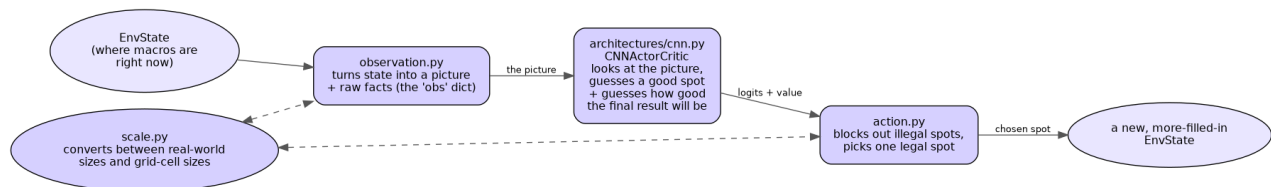
Before the file-by-file tour, here are five pictures showing how groups of functions actually call each other in a real training run.

Reading a chip file



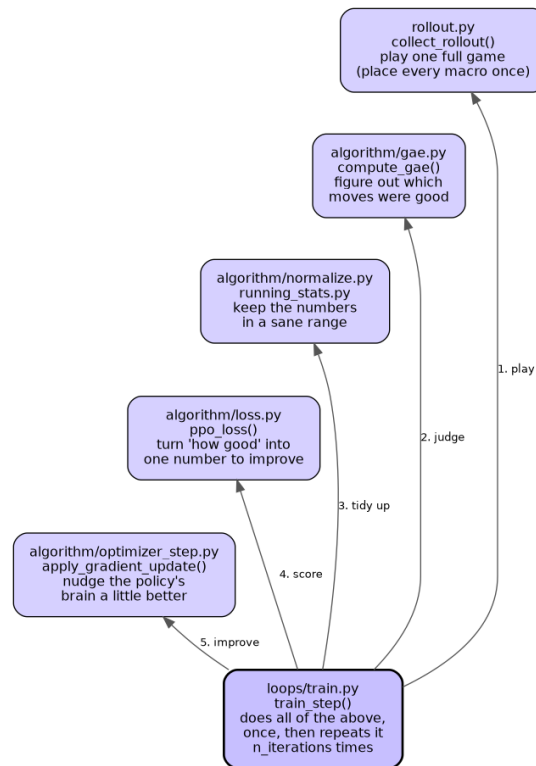
No matter which file format your chip blueprint is in, all three readers end up in the exact same simple shape — so everything downstream never has to care which format you started with.

Deciding where to place one macro



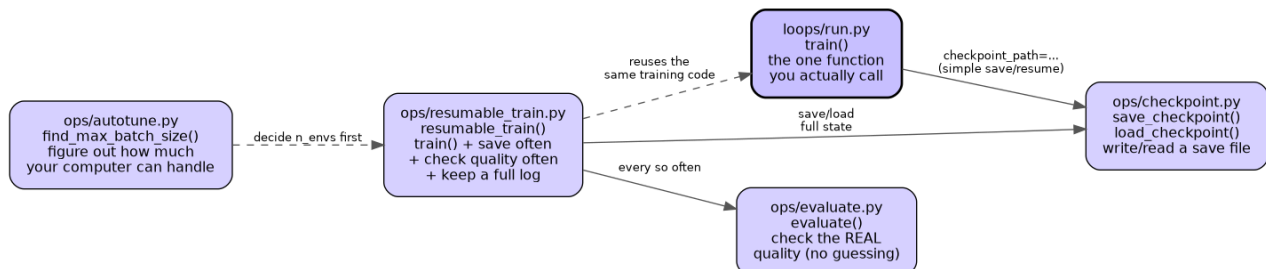
Every time it's a macro's turn, the game state becomes a picture, the AI brain looks at the picture, and `action.py` makes sure whatever it picks is actually legal.

One round of getting smarter



train_step() is the conductor — it calls all five of these, in this exact order, every single time.

Saving progress and checking real quality



train() can save on its own (simple), or resumable_train() can wrap it with periodic real quality checks and a permanent diary file (more powerful).

placax/

The engine room. This is the actual chip-placing machine: it knows how to read chip blueprints, keep track of where each piece is, and check if a piece was put in a silly spot. It doesn't know anything about learning or getting smarter - that's a different package's job.

(top level)

The core files that everything else in placax depends on.

core.py

The heart of placax - two tiny functions that do all the real work.

- **reset()**

Starts a brand new game. Clears the board so no macros are placed yet (or lets you start partway through, if you already know where some pieces go).

- **step()**

Places ONE macro at the spot you picked, then checks: are we done? If every macro is placed, it hands out a reward (a score for how good the whole layout is). If not, the reward is 0 - no cheating by getting paid early.

- **random_action()**

Picks a totally random spot on the grid. Not smart at all - just useful for quickly checking the game works, without needing a real AI player yet.

types.py

The shared shapes and labels every other file agrees to use, so everyone speaks the same language.

- **EnvState**

A little box holding 'where is every macro right now' and 'how many have we placed so far'. This changes every single step.

- **EnvParams**

A little box holding the RULES of the game that don't change during a game - how big the grid is, how many macros there are.

- **EnvParams.grid_x / effective_grid_y**

Two helper labels so the grid can be a rectangle (like 96 wide, 48 tall) instead of always being a perfect square.

- **RewardFn / SizeMap / NetPin / Nets / PinOffsets**

Just names for shapes of data, like labeling a box 'this box always has 2 socks in it' - so every function knows exactly what it's getting.

_device.py

Figures out what kind of computer we're running on (a slow CPU or a fast GPU) and quietly sets things up to run well on it.

- **_is_wsl()**

Checks: are we running inside Windows' 'Linux inside Windows' trick? Some settings need to be different there.

- **warn_if_gpu_unused()**

If your computer HAS a fast GPU but you're not using it, this politely taps you on the shoulder and says 'hey, you're leaving speed on the table'.

- **recommended_parallelism_mode()**

Answers one question: should we train one game at a time, or many games at once? On a slow computer, one at a time is actually FASTER (measured for real, not guessed) - this function tells you which to pick.

netlist/

A chip's blueprint can come in different file formats - this folder has a reader for each one, and they all turn into the exact same simple shape afterward, like translators that all output the same language.

`__init__.py`

The receptionist - looks at a folder of chip files, figures out which format it is, and calls the right reader.

- **`detect_format()`**

Sniffs a folder's files and guesses: is this Bookshelf, DEF, or Protobuf format?

- **`load_netlist()`**

The one function everyone actually calls - hands you back the chip's macro sizes and wire connections, no matter which file format they started in.

`bookshelf.py`

Reads the oldest, simplest chip-file format (.nodes and .nets files).

- **`parse_nodes()`**

Reads the file listing every macro and its size.

- **`parse_nets()`**

Reads the file listing which pins are wired together.

- **`load_bookshelf()`**

Runs both of the above and hands back the combined result.

`def_reader.py`

Reads the DEF format (paired with a LEF file that has the actual macro shapes).

- **`parse_components()`**

Finds every macro/cell mentioned in the DEF file and where it currently sits.

- **`parse_nets()`**

Finds every wire connection in the DEF file.

- **`resolve_macro_sizes()`**

DEF alone doesn't say how big things are - this looks each one up in the matching LEF file.

- **`resolve_net_pin_offsets()`**

Figures out exactly where on each macro a wire actually connects, not just the macro's center.

- **`load_def()`**

Runs all of the above together.

`def_writer.py`

The reverse of `def_reader` - takes placax's placement decisions and writes them back into a real DEF file.

- **`write_placed_def()`**

Takes an existing DEF file and a list of new positions, and hands back a new DEF file with just those positions updated - everything else in the file stays exactly the same.

lef.py

Reads LEF files, which describe exactly what shape a macro is and where its pins are, in real physical units.

- **parse_lef_sizes()**

Reads how big (width x height) every macro type is.

- **_parse_macro_pins()**

Reads exactly where a macro's pins sit, for one macro.

- **parse_lef_pin_offsets()**

Runs the pin-reading for every macro in the file.

protobuf_reader.py

Reads Google's Circuit Training format - a text version of a TensorFlow-style graph file.

- **_split_nodes()**

Chops the giant text file into one chunk per graph node.

- **parse_macros()**

Finds every macro and its size.

- **_macro_pin_entries()**

Finds every macro's pins and exactly where they sit.

- **parse_macro_pins()**

A simpler version of the above, just the (macro, x, y) list.

- **parse_nets()**

Figures out which pins are wired together - trickier here since this format hides the wiring a bit differently than the others.

- **load_protobuf()**

Runs everything above together.

padding.py

All the netlist readers can give back DIFFERENT-length lists (one net might have 2 pins, another 50) - this file pads everything to the SAME length so the math can run fast on a computer chip.

- **_build_sizes_array()**

Turns the macro-size dictionary into one big number table.

- **build_padded_arrays()**

The main function - takes everything from a netlist reader and turns it into neat, same-sized number tables.

- **_group_pins_by_macro()**

For each macro, lists every wire it's part of.

- **build_macro_net_index()**

Turns that list into another neat, padded number table - used later so we don't have to check EVERY wire, just the ones touching the macro we're placing right now.

extras/

Optional add-on tools. Nothing here is required - the basic game (reset/step) works fine without any of it. These are things you plug in only if you want them.

masks.py

Helps answer 'is this spot actually legal to place a macro?'

- **compute_occupied()**

Marks which single grid cells already have a macro's reference corner sitting on them.

- **occupancy_mask()**

Marks every spot that would make a NEW macro overlap an already-placed one.

- **boundary_mask()**

Marks every spot where a macro would stick out past the edge of the chip.

render.py

Turns 'where are all the macros' into a picture (a grid of true/false values) that a computer-vision-style AI brain can look at.

- **render()**

Draws every placed macro's rectangle onto a grid, so you get a simple black-and-white picture of the chip so far.

rewards.py

Figures out how GOOD a finished layout is, using a classic chip-design measurement called wirelength.

- **hpwl()**

Measures total wire length using the cheapest, simplest estimate: draw a box around each wire's pins and add up the box's width and height, for every wire.

- **_wiremask_baseline()**

A helper that figures out the wirelength of everything ALREADY placed, before we even think about the new macro.

- **wiremask()**

For the macro about to be placed, works out - for EVERY possible spot on the grid at once - how much extra wire length that spot would cost. Lets an AI see a whole heatmap of good and bad spots instantly.

- **make_hpwl_reward()**

Packages hpwl() up into a ready-to-use reward function that step() can call.

mst.py

A second, more careful (but much slower) way to estimate wire length.

- **_prim_mst_length()**

Runs a classic algorithm (Prim's) to find the shortest way to connect a wire's pins together, instead of just drawing a box around them.

- **mst_wirelength()**

Runs the above for every wire in the chip and adds it all up.

placax_agents/

The learner. Everything in here is about training an AI to get GOOD at placing macros - the policy (the AI's 'brain'), how it looks at the board, how it practices, and how it saves its progress.

(top level)

Shared labels used across the whole learner package.

types.py

Two labels describing 'what does a policy look like' and 'what does a way-of-seeing-the-board look like', so any AI brain or any way of seeing the board can be swapped in.

- **AlgorithmFn**

A label meaning 'anything that takes a look at the board and returns a decision plus a guess of how good things will end up'.

- **StateFn**

A label meaning 'anything that turns the current game into something a brain can look at'.

policy/

Everything about how the AI SEES the board and DECIDES where to place a macro.

observation.py

Turns the raw game state into everything a brain might want to look at.

- **observation()**

Builds a little bundle: a picture of the board so far, the size of the macro about to be placed, and a few raw facts, all in one dict.

action.py

Turns the brain's raw guesses into one, actually-legal, chosen spot.

- **legal_action_logits()**

Takes the brain's raw scores for every spot and blanks out (sets to 'never pick this') every spot that's illegal - overlapping or off the edge.

- **sample_action()**

Rolls the dice (weighted by the brain's scores) and picks one spot.

- **action_log_prob()**

Works out exactly how likely the brain was to pick the spot it picked - needed later for the training math.

scale.py

Converts between 'real-world chip units' (micrometers) and 'grid cell units' (0, 1, 2, 3...) - the two units don't naturally match up.

- **compute_grid_scale()**

Figures out how many real-world units fit inside one grid cell, based on how big the macros are and how big the grid is.

- **to_grid_units()**

Converts a macro's real size into 'how many grid cells wide/tall is this'.

- **to_real_centers()**

Converts a chosen grid spot back into a real-world position, matching the exact formula MaskPlace itself uses - this fixed a real bug where the two units were being mixed up incorrectly.

architectures/cnn.py

One specific AI brain design - a small image-recognition-style network.

- **CNNActorCritic**

Looks at the board picture through a few convolution layers (like tiny sliding magnifying glasses), then gives back two things: a score for every possible spot, and one single guess of 'how good will this whole game end up'.

training/ (top level files)

The main practice-and-glue files, directly inside training/.

rollout.py

Plays practice games from start to finish.

- **collect_rollout()**

Plays one ENTIRE game from start to finish, letting the current brain choose every single spot, and writes down everything that happened (what it saw, what it picked, how it went) so it can be studied afterward.

reward.py

Fixes a units mismatch before scoring a game.

- **make_scaled_hpwl_reward()**

Wraps the basic wirelength reward so it correctly converts grid spots into real-world units first - without this, the reward math would be quietly wrong.

training/algorithm/

The actual learning MATH - the part that decides how to nudge the brain to be a little smarter.

config.py

One dial-box for every learning setting.

- **PPOConfig**

One small box holding every learning 'dial' (how much to trust new info, how cautious to be, etc) so they can all be tuned together from outside, instead of being stuck as buried defaults.

gae.py

Judges which moves in a finished game were actually good.

- **compute_gae()**

Looks back over a finished game and works out, for every single move, 'was this move better or worse than expected?' - this is what actually teaches the AI which decisions to repeat.

normalize.py

Keeps the 'how good was this move' numbers well-behaved.

- **normalize_advantages()**

Rescales those 'better or worse than expected' numbers so they're always a similar, sane size, which keeps learning stable.

running_stats.py

Remembers the average reward size across many games.

- **RunningStats / update_running_stats() / normalize_with_stats()**

Keeps a running average and spread of the rewards seen SO FAR across many games, so raw reward numbers (which can be huge, like -300,000) can be squashed down to a normal size before the AI tries to learn from them.

loss.py

Turns a whole game into one number to improve.

- **ppo_loss()**

Turns 'how the game went' into ONE single number to make smaller - the smaller this number gets, the better the AI is getting at both picking good spots AND correctly predicting how good a game will be.

optimizer_step.py

Actually nudges the AI's brain.

- **apply_gradient_update()**

Takes that one number and actually nudges every single number inside the AI's brain a tiny bit in the direction that would have made the score better.

training/loops/

The actual 'run training many times' loops - the functions you actually call to train an AI.

train.py

The simple, one-game-at-a-time training loop.

- **train_step()**

Runs ONE full round: play a game, judge it, tidy the numbers, score it, nudge the brain. One call = one tiny bit smarter.

- **train_sequential()**

Calls train_step() over and over, one game at a time - the simple, reliable way to train, and the fastest way on a normal computer without a big graphics card.

parallel_train.py

The many-games-at-once training loop.

- **parallel_train_step() / train_parallel()**

The same idea, but plays SEVERAL games at once before nudging the brain - much faster on a good graphics card, much SLOWER on a normal computer (measured directly, not guessed).

run.py

The one function most people actually call.

- **train()**

Looks at your computer, decides sequential-or-parallel automatically, and just runs the right one. Can also auto-save your progress every round if you ask it to.

ops/

Things you do AROUND training, not part of the core learning loop itself - checking quality, saving progress, and figuring out how much your computer can handle.

evaluate.py

Checks how good the AI's BEST guess is, with no randomness at all - useful for an honest progress check.

- **evaluate()**

Plays one game where the AI always picks its single favorite spot (no dice-rolling), then reports the REAL wirelength of the result - a fair number to compare against published results.

checkpoint.py

Saving and loading a training run's progress to/from a file, like a video game save file.

- **save_checkpoint()**

Writes the AI's current brain (and anything else you give it) to a file.

- **load_checkpoint()**

Reads that file back.

- **load_pretrained_from_url()**

Downloads someone else's already-trained brain from the internet (and remembers it locally, so it only downloads once) instead of starting from scratch.

autotune.py

Figures out how much your specific computer can actually handle, by trying it for real instead of guessing.

- **find_max_batch_size()**

Tries running bigger and bigger batches until one fails (safely, without crashing your whole computer), then reports the biggest size that worked.

resumable_train.py

The 'grown-up' version of saving progress - not just save/resume, but ALSO periodic quality checks and a permanent history log.

- **resumable_train()**

Trains for a while, automatically picking up where it left off if you've run this before, occasionally checking the REAL quality (not just the training score), and keeping a running diary file of the whole journey.

placax_tools/

Outside helpers. Placax only places the BIG pieces (macros). Turning that into a full, real, checked chip needs other, much bigger programs - this package is just the handshake between placax and those programs.

(top level)

The generic rulebooks that any specific tool has to follow.

cell_placer.py

The rulebook for any standard-cell placer.

- **CellPlacer**

The RULE for 'anything that places the small standard cells': give it a layout with macros placed, and it must hand back a layout with the small cells ALSO placed. Any tool that follows this rule can be swapped in.

validator.py

The rulebook for any chip checker.

- **Validator / PPAResult**

The RULE for 'anything that checks a finished chip': give it a finished layout, and it must hand back real numbers - like chip area and how full the chip is.

dreamplace/

The default tool for placing the small standard cells.

cell_placer.py

Actually talks to the real DREAMPlace program.

- **build_dreamplace_config() / DREAMPlaceCellPlacer**

Writes the settings file DREAMPlace needs, then actually runs DREAMPlace on your computer to finish the job DREAMPlace-style.

openroad/

The default tool for checking a finished chip for real.

validator.py

Actually talks to the real OpenROAD program.

- **build_openroad_script() / parse_openroad_output() / OpenROADValidator**

Writes a small script telling OpenROAD what to check, runs OpenROAD, then reads its answer back into simple numbers placax can use.

scripts/

Small, standalone programs you run directly from the command line - not imported by anything else.

(top level)

Two ready-to-run helper scripts.

download_benchmarks.py

Downloads real, famous chip-design test files from the internet, so you have something real to practice on.

- **`_fetch_ispd05_netlist()`**

Downloads one of the classic ISPD 2005 test chips.

- **`_fetch_single_file()`**

Downloads one plain file (used for the other formats).

- **`download_benchmark()`**

The main function - picks the right download method for whichever benchmark you asked for.

compare_sequential_vs_parallel.py

Runs a REAL speed test on YOUR computer, so you know for certain (not guessed) whether one-game-at-a-time or many-games-at-once is faster for you.

- **`run_sequential()`**

Times playing several games one after another.

- **`run_parallel()`**

Times playing several games all at once.